

Simulador de Sistema de Arquivos com Journaling

Disciplina: Projeto de Sistemas Operacionais **Curso:** Ciencia da Computacao - UNIFOR

Aluno 1: Vitor Vargas

Link do GitHub: <https://github.com/VitorVVC/file-system-simulator>

Resumo

Este trabalho propoe o desenvolvimento de um simulador de sistema de arquivos para compreender como sistemas operacionais organizam arquivos e diretorios. O projeto foi desenvolvido em Java e implementa operacoes basicas como criar, apagar, renomear e copiar arquivos, alem de criar, apagar, renomear e listar diretorios.

O simulador utiliza um arquivo chamado dados.dat para representar o disco virtual do sistema de arquivos. Dessa forma, as operacoes nao criam pastas reais no computador do usuario. A estrutura de arquivos e diretorios existe dentro do simulador e e persistida no arquivo dados.dat.

Alem disso, o projeto implementa um mecanismo de Journaling por meio do arquivo journal.log, registrando as operacoes antes e depois de sua execucao. Esse mecanismo simula uma tecnica usada em sistemas de arquivos reais para aumentar a integridade dos dados.

Parte 1 - Introducao ao Sistema de Arquivos com Journaling

Um sistema de arquivos e a parte do sistema operacional responsavel por organizar, armazenar, localizar e manipular arquivos e diretorios. Ele permite que os dados sejam acessados de forma estruturada, transformando o armazenamento fisico em uma organizacao compreensivel para usuarios e programas.

Em sistemas reais, o sistema de arquivos define como os arquivos sao nomeados, onde ficam armazenados, quais permissoes possuem e como os blocos de dados sao controlados no disco. Sem esse gerenciamento, o armazenamento seria apenas uma sequencia de dados sem organizacao clara.

Journaling

Journaling e uma tecnica usada para registrar operacoes antes que elas sejam efetivamente aplicadas ao sistema de arquivos. A ideia principal e manter um historico das acoes importantes para que, em caso de falha, seja possivel identificar o que estava sendo feito.

Neste simulador, o Journaling funciona com o conceito de write-ahead logging: antes de executar uma operacao, o sistema escreve no journal.log uma entrada BEGIN. Se a operacao for concluida corretamente, o sistema registra COMMIT. Se ocorrer algum erro, registra ERROR.

Exemplo de log:

```
[2026-06-05 10:20:00] BEGIN CREATE_DIR - /docs  
[2026-06-05 10:20:00] COMMIT CREATE_DIR - /docs
```

Assim, o arquivo `journal.log` permite acompanhar o historico das operacoes realizadas sobre o sistema de arquivos virtual.

Parte 2 - Arquitetura do Simulador

O projeto foi organizado em classes Java simples, cada uma com uma responsabilidade especifica.

Estruturas de dados utilizadas

- VirtualFile: representa um arquivo virtual, contendo nome e conteudo.
- VirtualDirectory: representa um diretorio virtual, contendo listas de arquivos e subdiretorios.
- FileSystemSimulator: representa o sistema de arquivos em si e implementa os metodos principais.
- Journal: gerencia o registro das operacoes no arquivo journal.log.
- Main: implementa o modo Shell, permitindo que o usuario execute comandos pelo terminal.

A estrutura principal do sistema e uma arvore de diretorios. O diretorio raiz / pode conter arquivos e outros diretorios. Cada diretorio tambem pode conter seus proprios arquivos e subdiretorios.

Persistencia em dados.dat

O arquivo dados.dat funciona como o disco virtual do simulador. Ele armazena a arvore de diretorios e arquivos usando serializacao de objetos Java.

Isso significa que, ao encerrar o programa, o estado do sistema de arquivos e salvo. Ao abrir novamente, o simulador carrega o conteudo anterior de dados.dat.

Journaling implementado

Cada operacao de alteracao segue o fluxo:

1. Registrar BEGIN no journal.log.
2. Executar a operacao solicitada.
3. Salvar a nova estrutura no dados.dat.

4. Registrar COMMIT no journal.log.

5. Em caso de erro, registrar ERROR.

Esse fluxo simula a ideia de integridade dos sistemas de arquivos com journaling, onde as operacoes importantes ficam registradas antes de serem confirmadas.

Parte 3 - Implementacao em Java

Classe FileSystemSimulator

A classe FileSystemSimulator e a classe principal do simulador. Ela possui os metodos responsaveis pelas operacoes do sistema de arquivos:

- `createFile`: cria um arquivo virtual.
- `copyFile`: copia um arquivo virtual.
- `deleteFile`: apaga um arquivo virtual.
- `renameFile`: renomeia um arquivo virtual.
- `createDirectory`: cria um diretorio virtual.
- `deleteDirectory`: apaga um diretorio virtual.
- `renameDirectory`: renomeia um diretorio virtual.
- `listDirectory`: lista o conteudo de um diretorio.
- `tree`:
exibe a arvore completa do sistema.

Classe VirtualFile

Representa um arquivo dentro do sistema virtual. Possui nome e conteudo. Tambem possui um metodo `copy`, utilizado para gerar uma copia do arquivo com outro nome.

Classe VirtualDirectory

Representa um diretorio dentro do sistema virtual. Possui uma lista de arquivos e uma lista de subdiretorios. Essa estrutura permite simular uma hierarquia semelhante a de sistemas reais.

Classe Journal

Responsavel por registrar no arquivo `journal.log` o inicio, a conclusao ou erro de cada operacao. Essa classe concentra a logica do Journaling.

Classe Main

Responsavel por executar o simulador em modo Shell. O usuario digita comandos como `mkdir`, `write`, `cp`, `rm`, `ls`, `tree` e `journal`, e o programa executa as operacoes correspondentes.

Parte 4 - Instalacao e funcionamento

Requisitos

- Java JDK 17 ou superior.
- Terminal ou IDE Java, como IntelliJ IDEA, Eclipse ou VS Code.

Como compilar

Entre na pasta do projeto e execute:

```
javac src/*.java
```

Como executar

```
java -cp src Main
```

Ao iniciar, o simulador exibira um prompt:

```
fs>
```

Digite `help` para visualizar os comandos disponiveis.

Comandos disponiveis

<code>mkdir/docs</code>	-> cria diretorio
<code>rmdir/docs</code>	-> apaga diretorio
<code>mkdir/docs documentos</code>	-> renomeia diretorio
<code>touch/docs/a.txt</code>	-> cria arquivo vazio
<code>write/docs/a.txt Ola mundo</code>	-> cria arquivo com conteudo
<code>cp /docs/a.txt /docs/b.txt</code>	-> copia arquivo
<code>rm /docs/a.txt</code>	-> apaga arquivo
<code>mv /docs/a.txt novo.txt</code>	-> renomeia arquivo
<code>ls /docs</code>	-> lista diretorio
<code>tree</code>	-> mostra arvore completa
<code>journal</code>	-> mostra log de journaling
<code>exit</code>	-> sair

Exemplo de uso

```
fs> mkdir /docs
OK: CREATE_DIR - /docs

fs> write /docs/a.txt Ola mundo
OK: CREATE_FILE - /docs/a.txt

fs> cp /docs/a.txt /docs/b.txt
OK: COPY_FILE - /docs/a.txt -> /docs/b.txt

fs> ls /docs
```

```
Conteudo de /docs:
[FILE] a.txt (arquivo, 9 caracteres)
[FILE] b.txt (arquivo, 9 caracteres)

fs> journal
[2026-06-05 10:20:00] BEGIN CREATE_DIR - /docs
[2026-06-05 10:20:00] COMMIT CREATE_DIR - /docs
```

Resultados esperados

Espera-se que o simulador ajude a compreender a organizacao basica de um sistema de arquivos e a importancia do Journaling. Com o uso do dados.dat, o projeto simula um disco virtual persistente. Com o journal.log, e possivel visualizar o registro das operacoes realizadas, aproximando o projeto de conceitos utilizados em sistemas de arquivos reais.

O sistema atende as operacoes solicitadas no trabalho: copiar, apagar e renomear arquivos; criar, apagar, renomear e listar diretorios. Alem disso, implementa um modo Shell simples e um mecanismo de Journaling com registros de BEGIN, COMMIT e ERROR.

Estrutura do projeto

```
file-system-simulator/
■■■ README.md
■■■ dados.dat           # gerado automaticamente na execucao
■■■ journal.log         # gerado automaticamente na execucao
■■■ src/
    ■■■ Main.java
    ■■■ FileSystemSimulator.java
    ■■■ VirtualFile.java
    ■■■ VirtualDirectory.java
    ■■■ Journal.java
```

Conclusao

O desenvolvimento do simulador permitiu compreender de forma pratica como um sistema operacional pode organizar arquivos e diretorios por meio de estruturas internas. O uso do arquivo dados.dat reforca a ideia de que o simulador trabalha com um sistema de arquivos virtual, sem depender da criacao de pastas reais no computador.

A implementacao do Journaling foi a parte principal do projeto, pois demonstra como sistemas de arquivos podem registrar operacoes antes de confirma-las. Esse mecanismo e importante para aumentar a confiabilidade e facilitar a recuperacao em caso de falhas. Assim, o trabalho contribui para a compreensao de conceitos fundamentais de sistemas operacionais, especialmente gerenciamento de arquivos, persistencia e integridade de dados.